LAB6 예비 보고서

STM32_2 컴퓨터와 FPGA 통신 — FPGA 서보모터·SRAM·DAC

과목 전자전기컴퓨터설계실험 2

제출일 2026년 10월 26일

성명 김태이

학번 2023440158

1 보고서 범위와 결론

10월 26일 실험(STM32_2)은 두 부분이다. NUCLEO-G474RE에서는 PC와 MCU 사이(UART2), MCU와 FPGA 사이(UART3)의 직렬 통신으로 과제5(PC 입출력 확인), 과제6(MCU에서 FPGA로 보낸 값 확인), 과제7(UART 명령으로 FPGA LED 제어)을 작성한다. Combo II-DLD에서는 교재 실습 서보모터 제어, SRAM 읽기·쓰기, DAC를 시뮬레이션하고 보드에서 시연한다.

MCU 코드는 한 파일에 작성하고 매크로 STM32_2_TASK로 과제5와 과제6·7을 골라 각각 Arm GCC로 빌드했다. 같은 소스를 PC에서 실행해 PC 입력 바이트가 어떻게 출력되는지, UART3로 어떤 바이트가 나가는지 확인했다. 과제6·7의 FPGA 쪽은 LAB3의 UART 에코를 고쳐 만들었고, 테스트벤치는 MCU 호스트 시뮬레이션이 UART3로 보낸 바이트 파일을 그대로 읽어 직렬 신호로 넣는다. PC → MCU → FPGA → PC의 전체 흐름을 코드 수준에서 이어 본 것이다.

FPGA 네 설계(서보, SRAM, DAC, 과제6·7)는 각각 매뉴얼 또는 직접 작성한 테스트벤치와 파형 확인용 테스트벤치가 Icarus Verilog와 Vivado 시뮬레이터에서 모두 통과했다. DAC와 과제6·7은 bitstream까지 만들었다. 서보 출력과 SRAM 버스 핀은 아직 장비 교재에서 확인하지 못해 배치·배선까지만 했다.

사전 검증에서 찾은 것이 하나 있다. 매뉴얼의 서보 PWM 회로는 50 MHz에서 타이밍 여유가 +0.023 ns 뿐이다. 펄스 폭을 구하는 32비트 곱셈·나눗셈이 한 클록 안에 있기 때문이다. 나눗셈을 없앤 대비안은 같은 테스트벤치를 통과하고 여유가 +13.528 ns로 늘었다. 핀을 정한 뒤 타이밍이 깨지면 대비안을 쓴다.

항목	사전 검증	결과
과제5	Arm GCC 빌드, 호스트 3개 입력	'1' 'A' 'a' → 0x31 0x41 0x61
과제6	호스트 + FPGA TB 연결	3바이트 PC 출력 일치
과제7	호스트 + FPGA TB 연결	명령 9개 LED 일치
서보	TB 2개, 배치·배선	PASS, 핀 미정
SRAM	TB 2개, 배치·배선	PASS, 핀 미정
DAC	TB 2개, bitstream	PASS, 타이밍 충족

여기서 PASS와 호스트 시뮬레이션 결과는 논리가 맞다는 뜻이다. 실제 전송 속도, 점퍼선 결선, Serial Monitor 설정, 서보의 실제 각도, SRAM·DAC 칩의 시간 조건과 아날로그 출력은 포함하지 않으므로 각 장 끝의 표대로 보드에서 따로 확인한다.

2 실험 환경과 검증 방법

구분	사용한 것
MCU 보드	NUCLEO-G474RE (STM32G474RE, Cortex-M4)
MCU 프로젝트	Glaysia/ece2_474 v1.1.0
MCU 빌드	Arm GNU Toolchain 13.3.Rel1, CMake 3.24.2, Ninja 1.11.1
호스트 검증	MinGW-W64 gcc 9.5.0, HAL 대역 헤더
FPGA 보드	Combo II-DLD, xc7s75fgga484-1
FPGA 도구	Vivado 2025.2, Icarus Verilog 12.0

MCU 프로젝트는 LAB4·LAB5와 같은 태그 v1.1.0이다. LAB5 폴더를 lab6/stm32_lab으로 복사한 뒤 ece2_474_routine.c만 원본으로 되돌리고 이번 과제를 새로 작성했다. 과제5는 기본 Debug 프리셋으로, 과제6·7은 같은 설정에 -DSTM32_2_TASK=6만 더해 별도 폴더에 빌드했다.

호스트 시뮬레이션은 LAB5의 HAL 대역에 HAL_UART_Transmit을 더한 것이다. 보드용 ece2_474_routine.c를 고치지 않고 PC용 gcc로 컴파일한다. getchar는 PC 표준 입력을 그대로 읽으므로, Serial Monitor가 보낼 바이트를 파일로 만들어 넣으면 된다. printf는 표준 출력으로 나오고, UART3로 보낸 바이트는 한 줄에 하나씩 16진수 파일로 남는다. FPGA 과제6·7 테스트벤치가 이 파일을 읽어 9600 bps 직렬 신호(시뮬레이션에서는 한 비트 8클록)로 바꿔 넣는다.

FPGA 교재 실습 세 개는 매뉴얼의 설계 모듈과 테스트벤치를 실습별 폴더에 넣어 실행했다. 핵심 구간만 보여주는 tb_view.sv를 따로 작성했고, Vivado 파형 캡처는 한 단 폭에서 읽히도록 시간축을 반으로 잘라 두 줄로 배치했다.

3 소스 출처와 실행 스냅샷

항목	식별 정보
MCU 템플릿	https://github.com/Glaysia/ece2_474.git
태그·커밋	v1.1.0 · dd94f01047aebdece09b9620c66e9b179403e5c9
FPGA 코드	ece2_474 docs/FPGA_additional.md 5·6·8절, LAB3-07 uart.v
제출 저장소	https://github.com/tae2yi/UOS_ece2
LAB6 코드	master (보고서의 GitHub 주소가 가리키는 위치)
폴더	lab6/stm32_lab, stm32_2_hostsim, lab6_01_servo, lab6_02_sram, lab6_03_dac, lab6_04_uart_led

GitHub: https://github.com/tae2yi/UOS_ece2/tree/master/lab6

템플릿에서 바꾼 파일은 이번에도 Core/Src/ece2_474_routine.c 하나다. 원본 대비 바뀐 내용 전체는 아래 diff에 있다. 호스트 시뮬레이션의 입력·출력, FPGA 시뮬레이션 로그, Vivado 리포트도 같은 저장소에 있다.

diff: https://github.com/tae2yi/UOS_ece2/blob/master/lab6/report/pre/logs/stm32_routine.diff

STM32_2 UART 직렬 통신

프레임과 속도

UART는 클록 선 없이 데이터 선 하나로 바이트를 보내는 비동기 직렬 통신이다. 선은 쉬는 동안 HIGH이고, 시작 비트(LOW) 하나, 데이터 8비트(D0, 즉 LSB부터), 정지 비트(HIGH) 하나를 차례로 보낸다. 이 형식을 8N1(8비트, 패리티 없음, 정지 비트 1)이라 한다[2]. 송신과 수신이 약속한 속도(baud)로 각자 시간을 재므로 양쪽 설정이 같아야 한다.

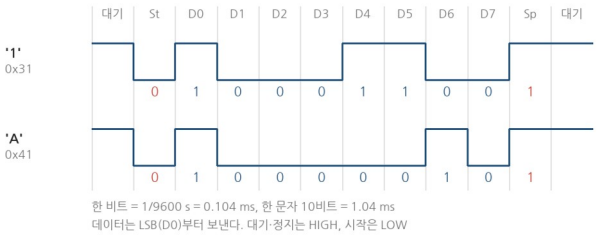


그림 1. 9600 bps 8N1에서 문자 '1'(0x31)과 'A'(0x41)의 선 파형. 시작 비트 다음 D0부터 나간다.

9600 bps에서 한 비트는 104.17 µs, 한 문자는 10비트이므로 1.04 ms다. 1초에 최대 960바이트를 보낸다. 받는 쪽은 시작 비트의 하강 에지를 보고 반 비트를 기다린 뒤, 한 비트 간격으로 각 비트의 가운데에서 값을 읽는다. 마지막 정지 비트의 가운데(시작에서 9.5비트 뒤)까지 두 쪽 시계의 누적 차이가 반 비트를 넘지 않으면 되므로, 속도 차이가 약 5 % 안이면 받을 수 있다.

쪽	분주	실제 속도	오차
STM32 USART2·3	100 MHz / 10417	9599.7 bps	-0.003 %
FPGA uart_rx·tx	50 MHz / 5208	9600.6 bps	+0.006 %

STM32의 USART2·3은 APB1 클록 100 MHz를 쓰고 16배 오버샘플링이므로 BRR = 100 MHz / 9600 = 10416.7 → 10417이다[1]. FPGA는 (50,000,000 + 4,800) / 9600 = 5208클록을 한 비트로 쓴다. 두 쪽 모두 0.01 % 안이라 속도 문제는 없다.

전기적 연결

STM32G474와 Combo II-DLD의 FPGA I/O는 모두 3.3 V 논리(LVCMOS33)이므로 PC10과 FPGA 입력을 점퍼선으로 바로 잇는다. PC의 RS-232 단자(±수 V)와는 전압이 달라 직접 연결하면 안 된다. 신호의 0 V 기준이 같아야 HIGH·LOW를 같은 뜻으로 읽으므로 두 보드의 GND를 반드시 함께 잇는다. USB 케이블이 둘 다 같은 PC에 꽂혀 있어도 GND 선을 생략하지 않는다.

코드 읽기 항목

표현	뜻
UART_HandleTypeDef	UART 설정·상태를 묶은 구조체 자료형. huart2·3은 그 변수
HAL_StatusTypeDef	enum. HAL_OK 0, ERROR 1, BUSY 2, TIMEOUT 3
int ch = getchar()	0~255에 더해 EOF(-1)도 담아야 해서 int
(uint8_t)ch	int를 1바이트 자료형으로 변환
&huart3, &data	장치 변수와 보낼 데이터의 주소(포인터)
1U, 100U	보낼 바이트 수 1, 최대 대기 100 ms

HAL_UART_Transmit은 송신 레지스터에 바이트를 넣고, 마지막 비트가 선으로 나갈 때까지 기다린 뒤 돌아온다[1]. 9600 bps 한 바이트면 약 1.04 ms 동안 이 함수 안에 머문다. 결과 HAL_OK는 MCU 쪽 송신이 끝났다는 뜻일 뿐이며, 선 반대쪽에 무엇이 연결되어 있는지는 알 수 없다. 점퍼선이 빠져 있어도 0이 나온다.

STM32_2 과제5 PC와 MCU 입출력 확인

요구 조건과 설계

· PC에서 보낸 문자 한 개를 MCU가 받고, 받은 문자와 그 바이트 값을 16진수로 VCP UART에 출력한다.

· '1', 'A', 'a'를 보내 0x31, 0x41, 0x61이 나오는지 확인한다. Serial Monitor의 줄바꿈은 없음으로 둔다.

· getchar가 입력을 기다리는 상태와 프로그램이 오류로 멈춘 상태를 구분한다.

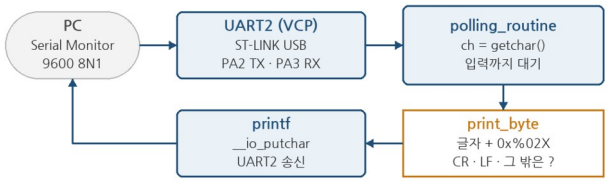


그림 2. 과제5의 처리 흐름. getchar가 UART2에서 한 바이트를 받을 때까지 기다리고, 받으면 글자와 16진수를 출력한다.

받은 바이트를 %c로 그대로 찍으면 줄바꿈 문자(CR 0x0D, LF 0x0A)는 화면에서 줄을 바꿔 버리고, 0x80 이상의 바이트는 깨진 글자가 된다. 그래서 print_byte 함수를 따로 두어 CR·LF는 C의 이스케이프 표기 두 글자로, 그 밖의 제어 문자와 0x7F 이상은 ?로 표시하고 16진수 값은 항상 함께 낸다. 과제6에서도 같은 함수를 쓴다.

핵심 코드

```
static void print_byte(uint8_t data)
{
  if (data == '\r')
  {
    printf("\\r");
  }
  else if (data == '\n')
  {
    printf("\\n");
  }
  else if (data < 0x20U || data > 0x7EU)
  {
    printf("?");
  }
  else
  {
    printf("%c", data);
  }
  printf(" (0x%02X)", data);
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/stm32_lab/Core/Src/ece2_474_routine.c#L63-L82

```
void polling_routine(void)
{
  /* PC에서 문자가 올 때까지 getchar 안에서 기다립니다. */
  int ch = getchar();
  if (ch == EOF)
  {
    return;
  }
  uint8_t data = (uint8_t)ch;
#if STM32_2_TASK == 5
  printf("받은 문자: ");
  print_byte(data);
  printf("\r\n");
#else
  HAL_StatusTypeDef result =
    HAL_UART_Transmit(&huart3, &data, 1U, 100U);
  printf("FPGA로 보냄: ");
  print_byte(data);
  printf(", 전송 결과: %d\r\n", (int)result);
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/stm32_lab/Core/Src/ece2_474_routine.c#L85-L105

과제5와 과제6·7은 #if STM32_2_TASK로 나눈다. 둘 다 getchar로 한 바이트를 받은 뒤, 과제5는 출력만 하고 과제6·7은 UART3로 보낸 뒤 결과를 출력한다. ece2_474_init()은 비워 둔다. UART2 인터럽트 수신을 시작하면 getchar와 같은 UART를 두 방식이 함께 읽게 되므로 매뉴얼대로 시작 호출을 넣지 않는다.

예상 결과

호스트 시뮬레이션에 세 가지 입력을 넣었다. 매뉴얼의 세 문자, 줄바꿈을 CR LF로 둔 채 A를 보낸 경우, 한글 "가"를 보낸 경우다. 출력은 다음과 같다.

```
$ printf '1Aa' | task5
받은 문자: 1 (0x31)
받은 문자: A (0x41)
받은 문자: a (0x61)
$ printf 'A\r\n' | task5
받은 문자: A (0x41)
받은 문자: \r (0x0D)
받은 문자: \n (0x0A)
$ printf '가' | task5   (UTF-8)
받은 문자: ? (0xEA)
받은 문자: ? (0xB0)
받은 문자: ? (0x80)
```

결과:
https://github.com/tae2yi/UOS_ece2/tree/master/lab6/stm32_2_hostsim/results

문자	16진수	2진수	비고
'1'	0x31	0011 0001	숫자 1(0x01)과 다름
'A'	0x41	0100 0001	
'a'	0x61	0110 0001	'A'와 비트 5만 다름
CR · LF	0x0D · 0x0A		줄바꿈 설정에 따라 붙음

ASCII에서 숫자 문자는 0x30부터, 대문자는 0x41부터, 소문자는 0x61부터 이어진다. 대문자와 소문자는 0x20(비트 5) 차이다. Serial Monitor의 줄바꿈 설정이 CR LF이면 문자 하나를 보낼 때마다 세 줄이 출력되므로, 한 문자 확인에서는 없음으로 둔다. 한글은 UTF-8에서 한 글자가 3바이트(가 = EA B0 80)이므로 세 번 출력된다.

기다림과 멈춤의 구분

getchar는 _read → __io_getchar → HAL_UART_Receive(…, HAL_MAX_DELAY)로 이어지므로, 입력이 없으면 이 안에서 끝없이 기다린다. 겉으로는 아무 출력도 없어 멈춘 것처럼 보인다. 둘을 구분하는 방법은 두 가지다. 첫째, 문자를 하나 더 보내 응답이 오면 기다리던 것이다. 둘째, 디버거에서 일시 정지(F6) 하면 호출 스택이 보인다. HAL_UART_Receive 안의 플래그 확인 반복에 있으면 기다리는 상태이고, HardFault_Handler나 Error_Handler의 무한 반복에 있으면 오류로 멈춘 것이다.

빌드 결과와 보드에서 확인할 항목

```
$ cmake --preset Debug
$ cmake --build --preset Debug
RAM:    2640 B / 128 KB  (2.01 %)
FLASH: 28028 B / 512 KB  (5.35 %)
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/report/pre/logs/stm32_arm_build.txt

확인 항목	예상 관찰
Serial Monitor 설정	9600, 8, None, 1, 줄바꿈 없음
'1' 'A' 'a' 전송	0x31, 0x41, 0x61 표시
줄바꿈 CR LF로 변경	0x0D, 0x0A 두 줄이 더 출력
한글 출력 깨짐	Serial Monitor 문자 인코딩 UTF-8 확인
입력 없이 일시 정지	호출 스택이 HAL_UART_Receive 안

STM32_2 과제6 MCU에서 FPGA로 보낸 값 확인

데이터 흐름과 배선

PC → VCP → MCU → 점퍼선 → FPGA → USB-UART → PC의 순서다. MCU는 PC에서 받은 한 바이트를 UART3(PC10)로 FPGA에 보내고, FPGA는 받은 바이트를 보드의 USB-UART로 PC에 그대로 보낸다. PC에는 COM 포트가 두 개 생긴다. MCU 포트로 문자를 보내고 FPGA 포트에서 같은 문자가 오는지 본다.

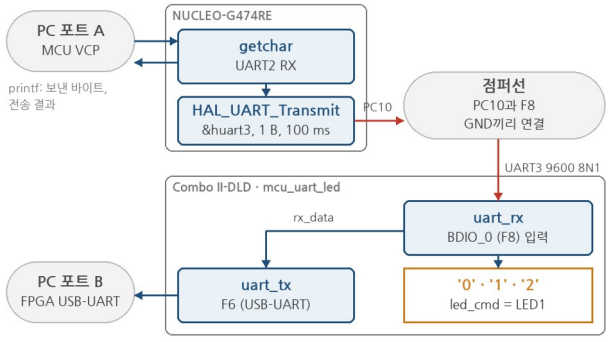


그림 3. 과제6·7의 연결. 빨간 선이 이번에 새로 잇는 점퍼선(MCU PC10 → FPGA BDIO_0, GND 공통)이다.

연결	MCU 쪽	FPGA 쪽
UART3 신호	PC10 (USART3 TX)	BDIO_0 = F8
접지	GND 핀	브레드보드 옆 J6 GND
PC 출력	-	F6 → USB-UART
응답(추가 구현)	PC11 (USART3 RX)	사용 안 함

FPGA 입력 핀은 브레드보드 I/O의 BDIO_0(F8)으로 정했다. 확장 포트 J19에는 신호등 모듈이 꽂혀 있고, 브레드보드 I/O는 옆에 GND 헤더가 있어 점퍼선 두 개로 연결하기 쉽다. 교재의 브레드보드 I/O 표에 따르면 이 핀들은 버스 스위치를 거치며 DIP 스위치 SW5(BD_SEL)를 켜야 FPGA와 이어지므로, 실험 전에 SW5를 확인한다. 3.3 V 신호이고 100 Ω 직렬 저항이 있다.

MCU 코드

앞의 polling_routine에서 #else 쪽이다. 매뉴얼의 제공 코드에 보낸 바이트 표시를 더했다. 결과는 FPGA로 보낸 문자, 16진수, HAL_UART_Transmit의 반환값 순서로 출력한다. 호스트 시뮬레이션 결과는 다음과 같고, UART3 파일에는 31 41 61이 남았다.

```
$ printf '1Aa' | task6 task6_chars_uart3.hex
FPGA로 보냄: 1 (0x31), 전송 결과: 0
FPGA로 보냄: A (0x41), 전송 결과: 0
FPGA로 보냄: a (0x61), 전송 결과: 0
```

UART3:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/stm32_2_hostsim/results/task6_chars_uart3.hex

FPGA 설계: LAB3 UART 에코에서 바꾼 것

LAB3-07의 uart_rx·uart_tx는 그대로 쓰고 최상위만 mcu_uart_led로 새로 만들었다. uart_rx는 입력을 플립플롭 두 개로 동기화한 뒤 시작 비트의 가운데를 확인하고 비트마다 한 번 읽는다. 정지 비트가 HIGH가 아니면 바이트를 버리고 framing_error를 낸다. 바뀐 것은 세 가지다. ① 수신 입력을 보드 USB-UART의 C6에서 점퍼선용 F8로 옮겼다. ② 과제7의 LED 명령 처리를 더했다. ③ LED1은 명령용으로 쓰고, 나머지 일곱 개에 마지막 바이트의 하위 7비트를 표시한다.

```
always @(posedge clk_50mhz) begin
  if (rst_p) begin
    tx_valid <= 0; tx_data <= 0; led_cmd <= 0; led_byte <= 0;
  end else begin
    tx_valid <= 0;
    if (rx_valid) begin
      led_byte <= rx_data[6:0];
      if (tx_ready) begin
        tx_data <= rx_data;
        tx_valid <= 1;
      end
      case (rx_data)
        8'h30: led_cmd <= 1'b0;      // '0'
        8'h31: led_cmd <= 1'b1;      // '1'
        8'h32: led_cmd <= ~led_cmd;  // '2'
        default: led_cmd <= led_cmd;
      endcase
    end
  end
end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_04_uart_led/src/mcu_uart_led.v#L33-L52

선을 뺐을 때 입력이 떠서 잡음이 시작 비트로 읽히지 않도록 XDC에서 F8에 내부 풀업을 켰다. UART 대기 상태가 HIGH라 정상 신호에는 영향이 없다. MCU는 PC 입력을 기다리는 동안 PC10을 HIGH로 유지한다.

파일	역할
src/uart.v	LAB3-07 uart_rx·uart_tx 그대로
src/mcu_uart_led.v	설계 top. 에코와 LED 명령
sim/tb.sv	MCU 출력 파일을 읽어 과제6·7 검사
sim/tb_view.sv	파형 확인용. '1' '2' '2' '0'
constraints/mcu_uart_led.xdc	F8 풀업 입력, F6, LED 8개

시뮬레이션 결과

tb.sv는 MCU 호스트 시뮬레이션이 만든 UART3 파일(과제6 3바이트, 과제7 9바이트)을 읽어 한 바이트씩 mcu_rxd에 직렬로 넣고, 동시에 uart_txd를 받아 PC로 나간 바이트가 같은지 비교한다. 정지 비트를 0으로 만든 잘못된 프레임은 버려야 한다는 것과 리셋도 검사한다. 9600 bps 대신 한 비트를 8클록으로 줄였다.

```
[Icarus] PASS uart_led: 12 bytes echoed,
    LED commands 0/1/2/other, framing error,
    reset (40171 ns)
[Vivado] PASS uart_led: ... (39931 ns)
[Vivado] PASS uart_led view: 1 2 2 0 -> led_cmd 1 0 1 0
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/master/lab6/report/pre/logs

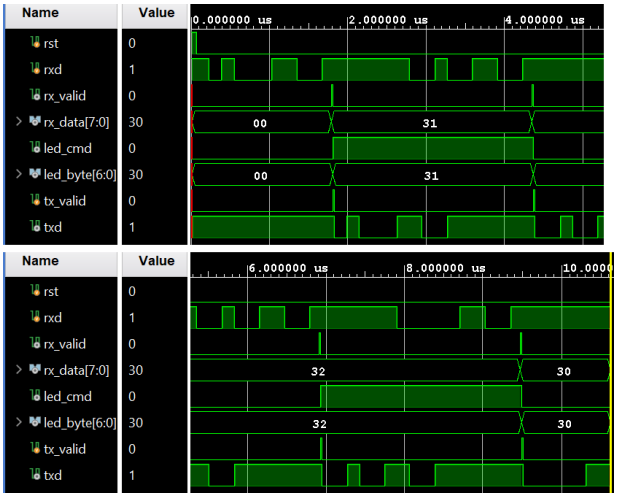


그림 4. 과제6·7 FPGA Vivado 파형(tb_view, 한 비트 8클록). rxd로 '1'(31)·'2'(32)·'2'·'0'(30)이 들어오면 rx_valid 다음 led_cmd가 1·0·1·0이 되고, 같은 바이트가 txd로 나간다.

rxd 파형에서 '1'(0x31)은 시작 비트 뒤 1, 0, 0, 0, 1, 1, 0, 0 순서로, 그림 1의 예상과 같다. rx_valid는 정지 비트의 가운데에서 한 클록 1이 되고, 그다음 클록에 tx_valid가 나가며 txd가 같은 비트열을 한 바이트 늦게 낸다.

구현 결과와 보드에서 확인할 항목

항목	값
LUT · 레지스터	136개 · 118개
타이밍	50 MHz에서 WNS +15.079 ns, 위반 없음
DRC	오류 0, 경고 1 (CFGBVS 미설정)
bitstream	lab6_04_uart_led/bitstream/mcu_uart_led.bit

XDC:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_04_uart_led/constraints/mcu_uart_led.xdc

확인 항목	예상 관찰
COM 포트 두 개	MCU VCP와 FPGA USB-UART가 따로 보임
MCU 포트에 '1'	MCU: 전송 결과 0 / FPGA 포트: 1
'A' 'a'	FPGA 포트에 A, a. LED2~8 = 100 0001 등
GND 선 제거	글자가 깨지거나 안 옴(비교용)
점퍼선 제거	MCU는 여전히 0, FPGA 포트는 무응답

STM32_2 과제7 UART 명령으로 FPGA LED 제어

MCU 코드는 과제6과 같다. FPGA가 받은 문자에 따라 LED1을 바꾼다. 비교 대상은 숫자가 아니라 문자이므로 '1'은 8'h31로 비교한다. 0x01과 비교하면 Serial Monitor에서 1을 보내도 아무 일이 일어나지 않는다.

문자	바이트	LED1(led_cmd)
'0'	0x30	끔
'1'	0x31	켬. 반복해도 켜짐 유지
'2'	0x32	반전. 보낼 때마다 바뀜
그 밖	-	유지

명령 처리는 앞의 always 블록의 case문이다. 반전은 레지스터 자신의 값을 뒤집어 다시 저장하는 것이므로, 같은 바이트가 한 번만 rx_valid로 오면 한 번만 바뀐다. uart_rx가 바이트마다 rx_valid를 한 클록만 내기 때문에 가능하다.

호스트 시뮬레이션에 '1122200x2'의 9바이트를 넣고, 그 UART3 출력 파일을 FPGA 테스트벤치로 넣은 결과는 다음과 같다.

```
task7 commands from MCU host simulation:
 1 0x31 -> PC 0x31, led_cmd=1
 1 0x31 -> PC 0x31, led_cmd=1
 2 0x32 -> PC 0x32, led_cmd=0
 2 0x32 -> PC 0x32, led_cmd=1
 2 0x32 -> PC 0x32, led_cmd=0
 0 0x30 -> PC 0x30, led_cmd=0
 0 0x30 -> PC 0x30, led_cmd=0
 x 0x78 -> PC 0x78, led_cmd=0
 2 0x32 -> PC 0x32, led_cmd=1
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/report/pre/logs/lab6_04_uart_led_tb_icarus.txt

'1'을 두 번 보내도 1을 유지하고, '2'를 세 번 보내면 0, 1, 0으로 매번 바뀐다. '0'을 두 번 보내도 0이고, x는 무시된다. 테스트벤치의 기대값 모델과 LED, PC로 나간 바이트가 모두 일치했다.

확인 항목	예상 관찰
'1' 여러 번	LED1 계속 켜짐
'2' 여러 번	보낼 때마다 켜짐·꺼짐 반복
'0'	LED1 꺼짐
x, A 등	LED1 그대로, FPGA 포트에는 문자 표시
줄바꿈 CR LF	LED 변화 없음(그 밖의 문자)

FPGA-01 서보모터 제어

회로 목적과 원리

RC 서보모터는 20 ms(50 Hz)마다 오는 펄스의 HIGH 폭으로 각도를 정한다. 내부 제어 회로가 펄스 폭을 기준 전압으로 바꾸고, 축에 달린 가변저항의 전압과 비교해 차이가 0이 되도록 모터를 돌린다[2]. 일반적으로 1 ms·1.5 ms·2 ms가 한쪽 끝·가운데·반대쪽 끝이다. 매뉴얼은 원본 예제의 0.7~2.3 ms 대신 1~2 ms 범위에서 버튼마다 0.1 ms씩 바꾼다.

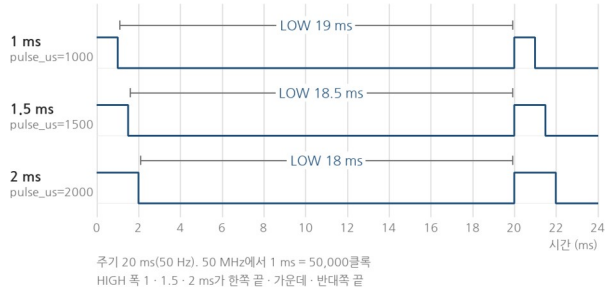


그림 5. 펄스 폭에 따른 서보 입력의 예상 파형. 주기는 20 ms로 같고 HIGH 폭만 바뀐다.

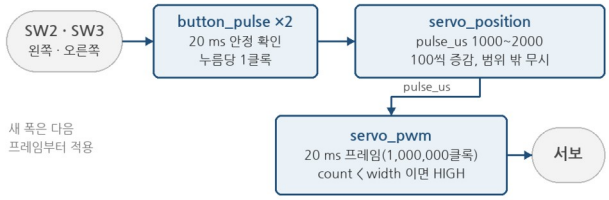


그림 6. FPGA-01 블록 흐름. 매뉴얼의 top은 servo_ctrl, 보드용 top은 버튼 처리를 더한 servo_board.

servo_position은 왼쪽·오른쪽 펄스마다 pulse_us를 100씩 줄이거나 늘리고 1000~2000 밖으로는 나가지 않는다. 두 펄스가 동시에 오면 무시한다. servo_pwm은 0부터 999,999까지 세는 프레임 카운터와 비교해 count < width_cycles인 동안 HIGH를 낸다.

```
  always @(posedge clk_50mhz) begin
    if(rst_p) begin
      count<=0; width_cycles<=(CLK_HZ/1000)*3/2;
    end
    else if(count==FRAME_CYCLES-1) begin
      count<=0;
      // 프레임 경계에서만 폭을 바꿔 중간 펄스가 잘리지 않게 합니다.
      width_cycles<=pulse_us*(CLK_HZ/1000)/1000;
    end
    else count<=count+1;
  end
```

GitHub: https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_01_servo/src/servo_pwm.v#L10-L20

width_cycles = pulse_us × (CLK_HZ/1000) / 1000 이므로 50 MHz에서 1500 µs는 75,000클록이다. 새 폭은 프레임 경계에서만 받으므로 버튼을 프레임 중간에 눌러도 진행 중인 펄스가 잘리지 않고, 다음 프레임부터 바뀐다.

버튼 처리

매뉴얼의 servo_ctrl은 버튼 한 번에 한 클록 길이의 펄스를 입력으로 받는다. 버튼의 원시 신호를 그대로 넣으면 누르고 있는 동안 50 MHz 클록마다 100씩 바뀌어 바로 끝까지 간다. 매뉴얼은 기존 실습 코드를 연결하라고 하므로, 동기화와 떨림 제거, 에지 검출을 한 모듈에 담은 button_pulse를 작성해 보드용 top servo_board에서 연결했다.

```
module button_pulse #(parameter STABLE_CYCLES=1_000_000)(
  input wire clk_50mhz,rst_p,button,
  output reg pulse
);
  (* ASYNC_REG="TRUE" *) reg [1:0] sync;
  reg stable;
  reg [31:0] count;
  always @(posedge clk_50mhz) begin
    if(rst_p) begin sync<=0; stable<=0; count<=0; pulse<=0; end
    else begin
      sync<={sync[0],button};
      pulse<=0;
      if(sync[1]==stable) count<=0;
      else if(count==STABLE_CYCLES-1) begin
        count<=0;
        stable<=sync[1];
        pulse<=sync[1];
      end
      else count<=count+1;
    end
  end
endmodule
```

GitHub: https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_01_servo/src/button_pulse.v#L5-L26

버튼 값이 20 ms(1,000,000클록) 동안 같은 값으로 유지될 때만 인정하고, 인정한 값이 0에서 1로 바뀌는 클록에 펄스를 한 번 낸다. 떨림은 보통 수 ms 안에 끝나므로 20 ms면 걸러진다. 반대로 20 ms보다 짧게 누르면 인정되지 않는다.

파일	역할
src/servo_position.v	매뉴얼. 펄스마다 ±100 µs
src/servo_pwm.v	매뉴얼. 20 ms 프레임 PWM
src/servo_ctrl.v	매뉴얼 top. 두 모듈 연결
src/button_pulse.v	작성. 떨림 제거·한 번 누름 펄스
src/servo_board.v	작성. 보드용 top
sim/tb.sv	매뉴얼 TB. 1·1.5·2 ms, 제한, 리셋

파일	역할
sim/tb_view.sv	작성. 떨리는 버튼부터 폭까지

시뮬레이션 결과

```
[Icarus] PASS servo: 20ms frame, 1/1.5/2ms pulse,
    limits, reset (100036 ns)
[Vivado] PASS servo: ... (100036 ns)
[Vivado] PASS servo view: bounce -> one step,
    widths 15/14/17/17 clocks (8035 ns)
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/master/lab6/report/pre/logs



그림 7. FPGA-01 Vivado 파형(tb_view, CLK_HZ=10000이라 한 프레임 200클록, 1 ms = 10클록). 떨리는 left 입력이 left_pulse 하나가 되어 1500→1400, right 세 번에 1700. 폭은 다음 프레임부터 15→14→17.

left가 1클록씩 세 번 떨린 뒤 눌려 있어도 left_pulse는 한 번만 나왔다. pulse_us는 누른 즉시 바뀌지만 width_cycles는 프레임 경계(count가 0으로 돌아갈 때)에서만 바뀌는 것이 파형에서 보인다.

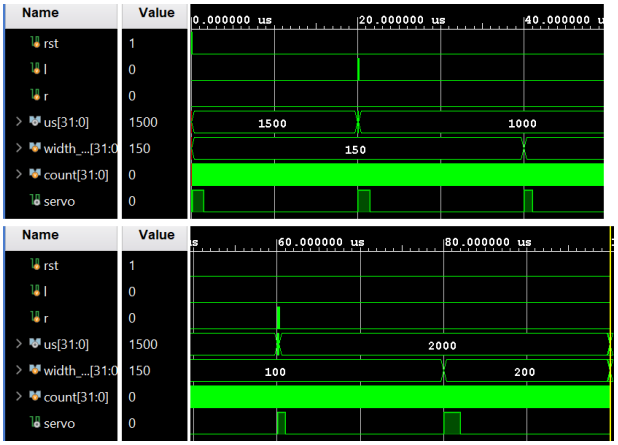


그림 8. FPGA-01 Vivado 전체 파형(매뉴얼 tb, CLK_HZ=100000). 폭 150 → 100(왼쪽 끝) → 200(오른쪽 끝)클록, 끝에서 리셋.

구현 결과와 타이밍 대비안

항목	매뉴얼 코드	대비안
LUT · 레지스터	466 · 121	115 · 127
WNS (50 MHz)	+0.023 ns	+13.528 ns
임계 경로	pulse_us → width_cycles	pulse_us 덧셈
논리 단수 · 지연	30단 · 19.91 ns	11단 · 6.47 ns

매뉴얼 코드는 타이밍을 충족했지만 여유가 0.023 ns뿐이다. 32비트 pulse_us에 50,000을 곱하고 1000으로 나누는 회로가 한 클록 안에 있어 CARRY4 15개를 포함한 30단 경로가 생겼다. 서보 핀을 정하면 배치가 달라지므로 위반이 날 수 있다.

```
    width_cycles<=(CLK_HZ%1_000_000==0)
      ? pulse_us*(CLK_HZ/1_000_000)
      : pulse_us*(CLK_HZ/1000)/1000;
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_01_servo/variants/servo_pwm_mul.v#L20-L22

대비안은 CLK_HZ가 1 MHz의 배수일 때 1 μs당 클록 수(50)를 곱하기만 한다. pulse_us가 최대 2000이면 $2000 \times 50{,}000 = 10^8$으로 32비트 안이므로 매뉴얼 식과 결과가 정확히 같다. 조건은 상수라 합성 때 한쪽만 남고, TB의 CLK_HZ=100000처럼 1 MHz의 배수가 아니면 매뉴얼 식을 그대로 쓴다. 대비안도 매뉴얼 TB와 tb_view를 통과했다.

신호	핀	근거
clk_50mhz · rst_p	B6 · K4	LAB3과 같음
button_left · right	N8 · N4	LAB5 피아노와 같은 SW2 · SW3
servo	미정	교재 서보 실습 핀 표 확인 필요

서보 출력 핀을 확인하지 못해 bitstream은 만들지 않았다. 핀이 정해지지 않은 포트가 있으면 Vivado가 DRC UCIO-1로 bitstream 생성을 막는다. 실험 전에 교재의 핀 표로 XDC 한 줄을 채우고 다시 빌드한다. 서보는 5 V 전원을 쓰지만 신호 입력은 3.3 V 펄스로 동작하는 것이 일반적이므로, 교재의 연결 회로를 함께 확인한다.

확인 항목	예상 관찰
리셋 뒤	가운데(1.5 ms) 위치
왼쪽 버튼 1번	한 단계(0.1 ms)만 회전
왼쪽 5번 이상	1 ms 끝에서 멈춤
오른쪽 10번 이상	2 ms 끝에서 멈춤
오실로스코프	주기 20 ms, HIGH 1~2 ms

FPGA-02 SRAM 읽기·쓰기

회로 목적과 원리

보드의 4 Mbit SRAM(256K × 16비트)에 주소와 데이터를 주어 쓰고, 같은 주소에서 읽어 LED로 확인한다. 비동기 SRAM은 클록이 없고 제어선 세 개로 동작한다. 모두 LOW일 때 활성(active low)이다[3].

동작	/CS	/OE	/WE	데이터 버스
쓰기	0	1	0	FPGA가 구동
읽기	0	0	1	SRAM이 구동, FPGA는 Z
대기	1	1	1	둘 다 Z

데이터 선은 쓰기와 읽기에 같이 쓰는 양방향 버스다. FPGA와 SRAM이 동시에 버스를 구동하면 한쪽은 HIGH, 다른 쪽은 LOW를 내는 순간 큰 전류가 흐르는 충돌이 생긴다. 그래서 FPGA는 쓸 때만 값을 내고 나머지는 3상태 버퍼로 고임피던스(Z)를 낸다[3].

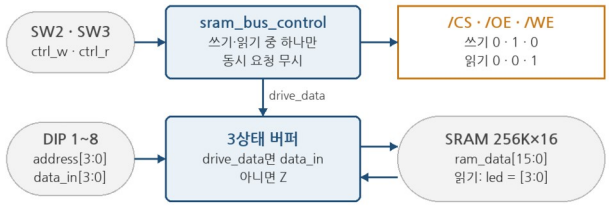


그림 9. FPGA-02 블록 흐름. 설계 top은 sram.

```
module sram_bus_control(
  input wire ctrl_w,ctrl_r,
  output wire drive_data,read_data,
  output wire ram_cs,ram_oe,ram_we
);
  assign drive_data=ctrl_w && !ctrl_r;
  assign read_data=ctrl_r && !ctrl_w;
  assign ram_cs=!(drive_data || read_data);
  assign ram_oe=!read_data;
  assign ram_we=!drive_data;
endmodule
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_02_sram/src/sram_bus_control.v#L4-L14

```
  assign ram_address={14'b0,address};
  assign ram_data=drive_data ? {12'b0,data_in} : 16'bz;
  assign led_out=read_data ? {4'b0,ram_data[3:0]} : 8'b0;
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_02_sram/src/sram.v#L12-L14

쓰기·읽기 요청이 동시에 들어오면 drive_data와 read_data가 모두 0이 되어 아무것도 하지 않는다(원본 예제와 다른 점). 주소는 하위 4비트만 쓰고 나머지 14비트는 0, 데이터는 16비트 중 하위 4비트만 쓴다. 읽을 때 LED에 ram_data[3:0]을 낸다.

시뮬레이션 결과

```
[Icarus] PASS SRAM: 16 addresses, bus release,
    simultaneous request (164 ns)
[Vivado] PASS SRAM: ... (164 ns)
[Vivado] PASS SRAM view: 4 writes, 4 reads (9 6 a 3),
    conflict ignored (360 ns)
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/master/lab6/report/pre/logs



그림 10. FPGA-02 Vivado 파형(tb_view). 주소 0~3에 9·6·A·3을 쓰고(wr) 다시 읽는다(rd). 쓰기·읽기가 아닐 때 bus는 ZZZZ, 마지막 동시 요청에서는 /CS·/OE·/WE가 모두 1.

쓰기 동안 bus는 FPGA가 낸 0009 등이고 나머지는 Z(파란색)다. 읽기 동안에는 SRAM 모델이 bus를 구동해 같은 값이 돌아오고 led에 나타난다. 이 TB의 SRAM 모델은 /WE 하강 직후 값을 저장하는 단순 모델이며, 실제 칩의 접근 시간(수십 ns)과 설정·유지 시간은 검사하지 않는다.

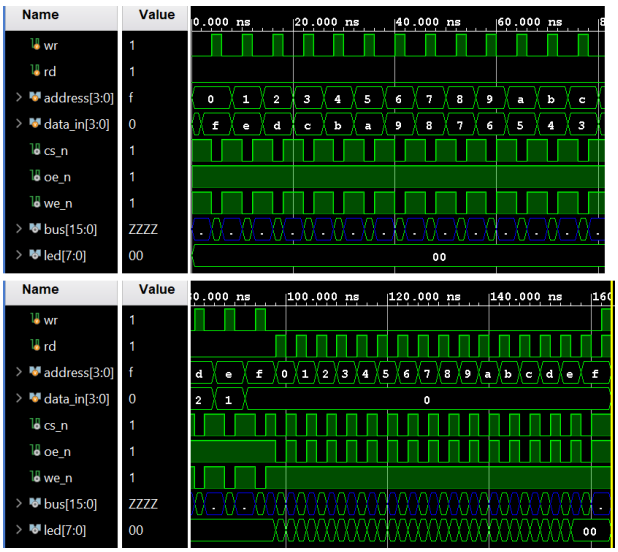


그림 11. FPGA-02 Vivado 전체 파형(매뉴얼 tb). 16개 주소에 15−i를 쓰고 읽은 뒤 동시 요청.

보드 동작에서 주의할 점

이 설계는 버튼을 누르는 동안 /WE를 LOW로 유지한다. 버튼을 누른 채 DIP 스위치로 주소를 바꾸면 /WE가 LOW인 동안 주소가 바뀌어 여러 주소에 쓰일 수 있다. 실제 SRAM의 쓰기 주기는 /WE가 LOW인 동안 주소가 안정되어 있어야 하기 때문이다. 실험에서는 주소와 데이터를 먼저 맞춘 뒤 버튼을 누르고, 뗀 다음에 스위치를 바꾼다. 버튼 떨림으로 /WE가 몇 번 오르내려도 같은 주소에 같은 값을 다시 쓰므로 결과는 같다.

16비트 SRAM에는 보통 상위·하위 바이트를 고르는 /UB·/LB 핀이 있다. 매뉴얼 설계에는 이 출력이 없으므로, 보드에서 이 핀이 FPGA에 연결되어 있으면 LOW로 고정하는 출력을 더해야 한다. 교재 핀 표에서 확인한다.

구현 결과와 보드에서 확인할 항목

항목	값
LUT · 레지스터	4개 · 0개 (조합 회로)
I/O 버퍼	OBUFT(3상태 출력) 16, IBUF 14
DRC	UCIO-1(핀 미정 포트), RPBF-3 12개, CFGBVS
bitstream	보류 (SRAM 핀 미정)

레지스터가 없어 타이밍 분석 대상 경로도 없다. RPBF-3은 ram_data[15:4]가 양방향 포트인데 입력 쪽을 쓰지 않아 버퍼가 반쪽뿐이라는 경고다. 설계상 상위 12비트는 읽지 않으므로 문제는 없다. 스위치·버튼·LED 핀은 LAB2·LAB5의 XDC와 같다.

확인 항목	예상 관찰
주소 0에 5 쓰기	주소 0, 데이터 5, SW2 누름
주소 0 읽기	SW3을 누르는 동안 LED = 0000 0101
다른 주소 읽기	쓴 적 없으면 알 수 없는 값
둘 다 누름	LED 꺼짐, 쓰기 안 됨
전원 껐다 켬	저장 값이 사라짐(휘발성)

FPGA-03 DAC

회로 목적과 원리

DAC는 디지털 값 N을 전압으로 바꾼다. 8비트 DAC의 출력은 기준 전압 V_REF에 대해 Vout ≈ V_REF × N / 256이고, 1 LSB는 V_REF / 256이다. 예를 들어 출력 범위가 0~3.3 V이면 한 단계가 약 12.9 mV다[2]. 보드의 DAC는 8비트 병렬 입력이며 /CS, /WR, A/B(채널), /LDAC(출력 갱신) 핀을 가진다. 칩 이름과 기준 전압은 교재로 확인할 항목이다.

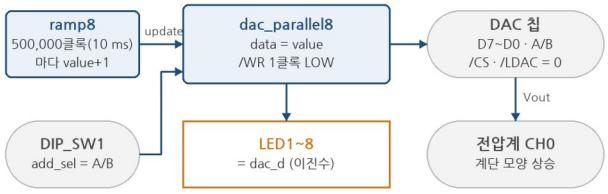


그림 12. FPGA-03 블록 흐름. 설계 top은 dac.

ramp8은 500,000클록(10 ms)마다 update를 한 클록 내고 값을 1 올린다. dac_parallel8은 update가 오면 그 값을 data에 등록하고 /WR을 한 클록(20 ns) LOW로 만든다. /CS와 /LDAC는 0으로 고정해 칩을 항상 선택하고, 쓴 값이 바로 출력에 반영되게 한다.

```
  always @(posedge clk_50mhz) begin
    if(rst_p) begin we_n<=1; a_b<=0; data<=0; end
    else begin
      we_n<=1;
      if(update) begin data<=value; a_b<=channel; we_n<=0; end
    end
  end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_03_dac/src/dac_parallel8.v#L12-L18

data와 /WR이 같은 클록 에지에서 바뀌고, 다음 에지에서 /WR만 HIGH로 돌아간다. 병렬 DAC는 보통 /WR 상승 에지에서 데이터를 래치하므로 데이터는 /WR LOW 동안(20 ns) 안정되어 있다. 칩이 요구하는 최소 /WR 폭과 데이터 설정 시간이 20 ns 이하인지 설명서로 확인하고, 모자라면 /WR LOW를 여러 클록으로 늘린다.

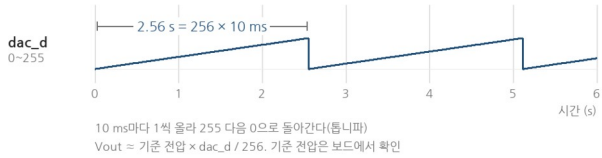


그림 13. DAC 입력 값의 예상 변화. 2.56초 주기의 톱니파가 되고 출력 전압도 같은 모양이다.

256단계 × 10 ms = 2.56초마다 0에서 최대까지 올라갔다가 0으로 떨어진다. 전압계로 보기에 충분히 느리고, LED에도 같은 값이 이진수로 표시되어 매초 약 100씩 증가한다.

시뮬레이션 결과

```
[Icarus] PASS DAC: 260 writes, wraparound, channel,
    data hold, reset (10436 ns)
[Vivado] PASS DAC: ... (10436 ns)
[Vivado] PASS DAC view: 12 writes (0..11),
    channel A then B (530 ns)
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/master/lab6/report/pre/logs

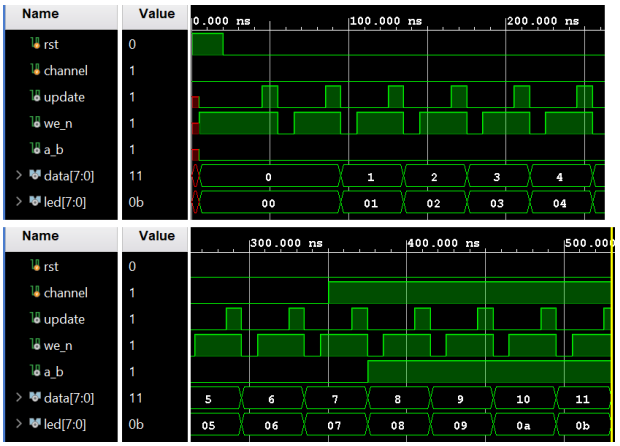


그림 14. FPGA-03 Vivado 파형(tb_view, UPDATE_CYCLES=4라 5클록마다 쓰기). we_n 펄스마다 data가 0, 1, 2…로 늘고, channel을 바꾼 뒤의 쓰기부터 a_b=1.

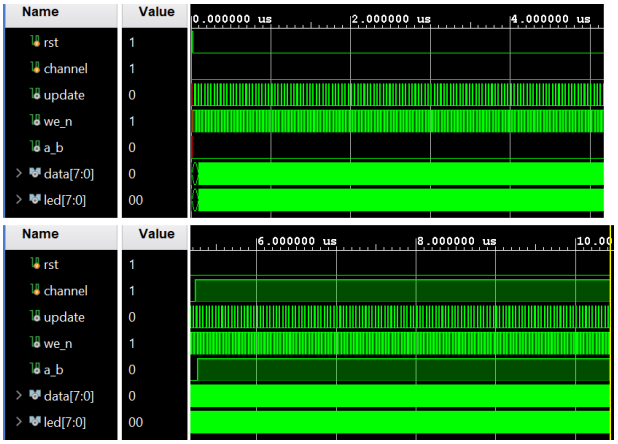


그림 15. FPGA-03 Vivado 전체 파형(매뉴얼 tb). 260번 쓰는 동안 255 다음 0으로 돌아가고 130번째부터 채널 B.

핀 배치와 구현 결과

신호	핀	근거
dac_csn · ldacn	E1 · F4	교재 DAC 실습 표
dac_wen · a_b	G4 · G2	교재 DAC 실습 표
dac_d[7:0]	B3 B1 C4 C2 D5 D2 E5 E3	교재 D7~D0
add_sel	Y1	교재 DIP_SW1
led_out[7:0]	L4 M4 M2 N7 M7 M3 M1 N5	교재 LED1~8

교재 표의 clk 핀은 N8로 적혀 있지만 N8은 다른 실습에서 버튼 SW2로 쓰인 핀이라 오타로 보고, 메인 클록 B6을 썼다.

항목	값
LUT · 레지스터	19개 · 58개
타이밍	50 MHz에서 WNS +14.824 ns, 위반 없음
DRC	오류 0, 경고 1 (CFGBVS 미설정)
bitstream	lab6_03_dac/bitstream/dac.bit

XDC:
https://github.com/tae2yi/UOS_ece2/blob/master/lab6/lab6_03_dac/constraints/dac.xdc

보드에서 확인할 항목

확인 항목	예상 관찰
전압계 CH0	0 V에서 2.56초 동안 올라 다시 0 V
LED1~8	이진수 카운트. LED1(MSB)은 1.28초마다 바뀜
LED8(LSB)	10 ms마다 바뀌어 반쯤 켜진 듯 보임
DIP_SW1 = 1	CH0(채널 A)은 마지막 값에서 멈춤
최대 전압	기준 전압 × 255/256 기록
오실로스코프	/WR 10 ms 주기, LOW 20 ns

4 예상 결과 정리와 실험 계획

이번 주 실습의 공통점은 정해진 시간 형식이다. UART는 104 µs마다 한 비트를, 서보는 20 ms마다 1~2 ms 펄스를, DAC는 10 ms마다 한 번의 쓰기 펄스를 낸다. FPGA에서는 모두 50 MHz 클록을 세는 카운터로 이 시간을 만든다. SRAM만 클록 없이 제어선의 조합으로 동작하며, 그 대신 버스를 누가 구동하는지가 중요하다.

순서	할 일	기록할 것
1	과제5 다운로드	'1' 'A' 'a' 출력 화면
2	PC10·GND 점퍼선, SW5	배선 사진
3	mcu_uart_led.bit, 과제6	두 COM 포트 화면
4	과제7 명령	LED1 변화 영상
5	서보·SRAM XDC 완성	교재 핀 표, 빌드 결과
6	서보 시연	버튼별 각도, PWM 파형
7	SRAM·DAC 시연	LED 값, 전압계 값

실험 후 보고서에서는 위 결과를 이 보고서의 예상 출력과 비교한다. 서보 핀을 정한 뒤 매뉴얼 코드의 타이밍이 깨지는지와 그때 대비안을 썼는지, SRAM의 /UB·/LB 처리, DAC의 최대 전압을 함께 적는다.

참고 문헌

[1] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal_uart.c). https://github.com/STMicroelectronics/STM32CubeG4

[2] Sarah L. Harris and David Harris, Digital Design and Computer Architecture: ARM Edition, Morgan Kaufmann, 2015. ISBN 978-0-12-800056-4.

[3] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018. ISBN 978-0-13-454989-7.